27.1 节点概念

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~250k tokens，输出 ~6k tokens（本节）

一台运行 OpenClaw Gateway 的电脑，能力是有限的——它可以执行 Shell 命令、读写文件、控制浏览器，但它**看不到手机摄像头**、**无法推送系统通知到 iPad**、**也不能调用远程服务器上的 GPU 集群**。OpenClaw 的节点系统（Node System）就是为了解决这个问题：让 Agent 可以**跨设备调用能力**，就像在本地执行一样自然。

本节介绍节点系统的核心概念：什么是节点、节点有哪些角色、以及节点命令的命名空间与分类。

27.1.1 什么是节点：设备能力的远程暴露

基本概念

在 OpenClaw 的术语中，**节点（Node）** 是指通过 WebSocket 连接到 Gateway 的一个远程设备或进程。节点的核心职责是将自身的硬件和软件能力**暴露（expose）** 给 Gateway，使得 Agent 可以像调用本地工具一样远程使用这些能力。

> **衍生解释：能力暴露（Capability Exposure）**
>
> "能力暴露"是分布式系统中的一个常见模式。类似于操作系统中的设备驱动将硬件能力抽象为文件操作（Unix 的"一切皆文件"理念），节点系统将远程设备的能力抽象为**命令调用**。Agent 发出 `camera.snap` 命令，不需要知道摄像头连接在哪台设备上——Gateway 会自动路由到拥有摄像头的节点。

节点的数据模型

每个连接到 Gateway 的节点都由 `NodeSession` 类型描述：

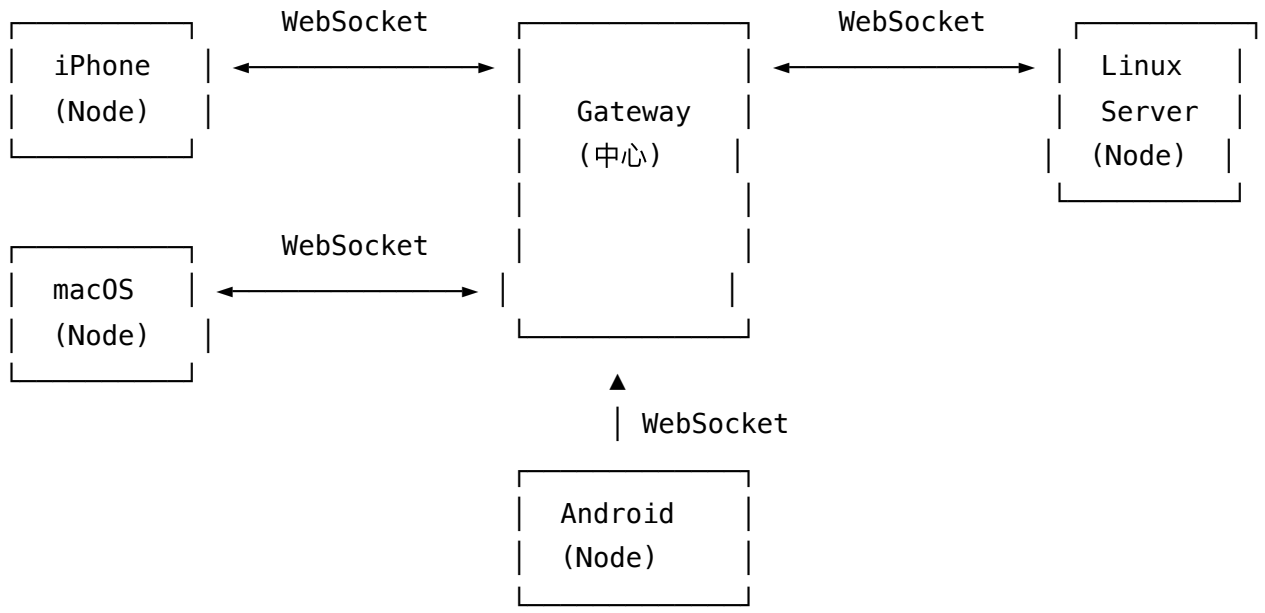
```ts
// src/gateway/node-registry.ts

export type NodeSession = {
  nodeId: string;           // 节点唯一标识（设备 ID 或客户端 ID）
  connId: string;           // 当前 WebSocket 连接 ID
  client: GatewayWsClient; // WebSocket 客户端实例
  displayName?: string;     // 人类可读名称，如 "MacBook Pro"
  platform?: string;        // 操作系统标识，如 "ios"、"darwin"、"android"
  version?: string;         // 客户端版本号
  coreVersion?: string;     // OpenClaw 核心版本
  uiVersion?: string;       // UI 版本（仅原生客户端）
  deviceFamily?: string;    // 设备系列，如 "iPhone"、"iPad"
  modelIdentifier?: string;// 设备型号，如 "iPhone15,2"
  remoteIp?: string;        // 远程 IP 地址
  caps: string[];           // 能力列表，如 ["system", "browser"]
  commands: string[];       // 声明支持的命令列表
  permissions?: Record<string, boolean>;  // 权限状态
  pathEnv?: string;         // PATH 环境变量
  connectedAtMs: number;    // 连接时间戳
};
```

这个类型揭示了节点系统的设计哲学：**节点是自描述的**。连接时，节点向 Gateway 报告自己的身份（`nodeId`、`platform`）、能力（`caps`、`commands`）和元数据（`version`、`deviceFamily`），Gateway 据此决定可以向该节点下发哪些命令。

连接架构

节点与 Gateway 之间的连接拓扑是经典的**星型结构（Star Topology）**：



所有节点都连接到同一个 Gateway，形成中心化的管理模式。这种架构的优势在于：

特性	说明
统一路由	Gateway 知道所有在线节点，可以选择最合适的目标
权限集中	命令白名单在 Gateway 侧统一配置和检查
状态感知	Gateway 实时感知哪些节点在线，能力如何
简化节点	节点只需要实现"接收命令并返回结果"的逻辑

27.1.2 节点角色：macOS / iOS / Android / 无头

平台与角色

不是所有节点都提供相同的能力。iPhone 有摄像头但没有 Shell 环境；Linux 服务器可以执行任意命令但没有 GUI。OpenClaw 通过**平台标识**来区分不同角色的节点，并为每个平台预设允许的命令集。

平台识别由 `normalizePlatformId` 函数完成：

```ts
// src/gateway/node-command-policy.ts

function normalizePlatformId(platform?: string, deviceFamily?: string): string {
  const raw = (platform ?? "").trim().toLowerCase();
  if (raw.startsWith("ios"))     return "ios";
  if (raw.startsWith("android")) return "android";
  if (raw.startsWith("mac") || raw.startsWith("darwin")) return "macos";
  if (raw.startsWith("win"))     return "windows";
  if (raw.startsWith("linux"))   return "linux";
  // 回退：根据 deviceFamily 推断
  const family = (deviceFamily ?? "").trim().toLowerCase();
  if (family.includes("iphone") || family.includes("ipad")) return "ios";
  if (family.includes("android")) return "android";
  // ... 更多回退逻辑
  return "unknown";
}
```

这个函数采用**两级回退策略**：先看 `platform` 字符串前缀，如果无法识别再根据 `deviceFamily` 推断。这样即使节点报告了非标准的平台字符串，也能正确分类。

四种典型角色

根据平台和用途，节点通常分为以下四类：

角色	平台	典型能力	使用场景
macOS 桌面	darwin	Shell 执行、浏览器代理、Canvas、摄像头	作为主力开发和执行节点
iOS 移动	ios	Canvas、摄像头、录屏、定位	移动端交互、拍照识图
Android 移动	android	Canvas、摄像头、录屏、定位、短信	同 iOS，额外支持 SMS
无头服务器	linux/windows	Shell 执行、浏览器代理	远程计算、CI/CD

每种角色的**默认命令允许集**在 `PLATFORM_DEFAULTS` 中硬编码：

```
// src/gateway/node-command-policy.ts

const PLATFORM_DEFAULTS: Record<string, string[]> = {
  ios:     [...CANVAS_COMMANDS, ...CAMERA_COMMANDS, ...SCREEN_COMMANDS, ...LOCATION_COM
  android: [...CANVAS_COMMANDS, ...CAMERA_COMMANDS, ...SCREEN_COMMANDS,
            ...LOCATION_COMMANDS, ...SMS_COMMANDS],
  macos:   [...CANVAS_COMMANDS, ...CAMERA_COMMANDS, ...SCREEN_COMMANDS,
            ...LOCATION_COMMANDS, ...SYSTEM_COMMANDS],
  linux:   [...SYSTEM_COMMANDS],
  windows: [...SYSTEM_COMMANDS],
  unknown: [/* 所有命令的并集——保守地允许一切 */],
};
```

`unknown` 平台被赋予了**最宽松的权限**——所有命令类型的并集。这看起来反直觉，但它确保未知设备类型的功能不会被意外屏蔽。实际的安全控制依赖于节点的 `commands` 声明和配置层面的 `allowCommands` / `denyCommands` 。

移动节点检测

Gateway 在某些场景下需要知道是否有移动设备在线（例如，决定是否提供拍照工具）。这由一个简洁的辅助函数实现：

```
// src/gateway/server-mobile-nodes.ts

const isMobilePlatform = (platform: unknown): boolean => {
  const p = typeof platform === "string" ? platform.trim().toLowerCase() : "";
  return p.startsWith("ios") || p.startsWith("ipados") || p.startsWith("android");
};

export function hasConnectedMobileNode(registry: NodeRegistry): boolean {
  return registry.listConnected().some((n) => isMobilePlatform(n.platform));
}
```

27.1.3 节点命令：命名空间与分类

命令命名空间

节点命令采用**点分命名空间（dot-separated namespace）** 组织，类似于 Java 的包名或 DNS 域名。每个命名空间对应一类设备能力：

命名空间	命令	说明
`canvas.*`	`canvas.present` ， `canvas.hide` ， `canvas.navigate` ， `canvas.eval` ， `canvas.snapshot` ， `canvas.a2ui.push` ， `canvas.a2ui.pushJSONL` ， `canvas.a2ui.reset`	可视化工作区操作
`camera.*`	`camera.list` ， `camera.snap` ， `camera.clip`	摄像头控制
`screen.*`	`screen.record`	录屏
`location.*`	`location.get`	地理定位
`sms.*`	`sms.send`	短信发送（仅 Android）
`system.*`	`system.run` ， `system.which` ， `system.notify` ， `system.execApprovals.get` ， `system.execApprovals.set`	系统命令执行
`browser.*`	`browser.proxy`	浏览器代理

这种命名约定使得权限控制非常灵活——可以按命名空间整体授权（如允许所有 `canvas.*` ），也可以精确到单个命令。

命令的三层控制

一个命令能否被成功执行，需要通过三层检查：

```
┌──────────────────────────────────────────────────────┐
│  第一层：平台默认允许集（PLATFORM_DEFAULTS）          │
│  + 配置追加（allowCommands）                          │
│  − 配置移除（denyCommands）                           │
│  → 得到 allowlist                                     │
├──────────────────────────────────────────────────────┤
│  第二层：节点自身声明（commands 数组）                 │
│  → 节点必须在连接时声明自己支持该命令                  │
├──────────────────────────────────────────────────────┤
│  第三层：Gateway 侧策略检查                            │
│  → isNodeCommandAllowed() 综合判断                    │
└──────────────────────────────────────────────────────┘
```

这个设计实现了**双向约束**：

1. **Gateway 侧约束**：即使节点声称支持某个命令，如果 Gateway 配置不允许，该命令也无法执行。
2. **节点侧约束**：即使 Gateway 允许某个命令，如果节点没有声明支持，也会被拒绝。

具体实现在 `isNodeCommandAllowed` 函数中：

```typescript
// src/gateway/node-command-policy.ts

export function isNodeCommandAllowed(params: {
  command: string;
  declaredCommands?: string[];
  allowlist: Set<string>;
}): { ok: true } | { ok: false; reason: string } {
  const command = params.command.trim();
  if (!command) {
    return { ok: false, reason: "command required" };
  }
  // 检查 1：是否在 allowlist 中
  if (!params.allowlist.has(command)) {
    return { ok: false, reason: "command not allowlisted" };
  }
  // 检查 2：节点是否声明支持
  if (Array.isArray(params.declaredCommands) && params.declaredCommands.length > 0) {
    if (!params.declaredCommands.includes(command)) {
      return { ok: false, reason: "command not declared by node" };
    }
  } else {
    return { ok: false, reason: "node did not declare commands" };
  }
  return { ok: true };
}
```

而 allowlist 的构建由 resolveNodeCommandAllowlist 完成：

```ts
// src/gateway/node-command-policy.ts

export function resolveNodeCommandAllowlist(
  cfg: OpenClawConfig,
  node?: Pick<NodeSession, "platform" | "deviceFamily">,
): Set<string> {
  const platformId = normalizePlatformId(node?.platform, node?.deviceFamily);
  const base = PLATFORM_DEFAULTS[platformId] ?? PLATFORM_DEFAULTS.unknown;
  const extra = cfg.gateway?.nodes?.allowCommands ?? [];        // 配置追加
  const deny = new Set(cfg.gateway?.nodes?.denyCommands ?? []); // 配置移除
  const allow = new Set([...base, ...extra]);
  for (const blocked of deny) {
    allow.delete(blocked.trim());
  }
  return allow;
}
```

> **衍生解释：白名单（Allowlist）模式**
>
> 安全领域有两种基本策略：黑名单（Blocklist，默认允许，列出禁止项）和白名单（Allowlist，默认禁止，列出允许项）。节点命令系统采用白名单模式——先从平台默认集合出发，然后通过 `allowCommands` 追加、 `denyCommands` 移除，最终得到精确的允许集合。白名单模式在安全上更加保守，因为遗漏一个规则只会导致功能不可用（安全但不便），而非功能泄露（危险但方便）。

命令调用时序

当 Agent 需要调用一个节点命令时（例如 `camera.snap` ），完整的时序如下：

```
Agent                    Gateway                    Node (iPhone)
  |                         |                           |
  |  node.invoke            |                           |
  |  {nodeId, command:      |                           |
  |   "camera.snap",        |                           |
  |   params: {...}}        |                           |
  |------------------------>|                           |
  |                         |                           |
  |                         |  ① 查找节点是否在线        |
  |                         |  ② 检查命令是否允许        |
  |                         |  (isNodeCommandAllowed)   |
  |                         |                           |
  |                         |  node.invoke.request      |
  |                         |  {id, command, paramsJSON} |
  |                         |-------------------------->|
  |                         |                           |
  |                         |                           |  执行拍照
  |                         |                           |
  |                         |  node.invoke.result       |
  |                         |  {id, ok: true,           |
  |                         |   payloadJSON: "..."}     |
  |                         |<--------------------------|
  |                         |                           |
  |  {ok: true,             |                           |
  |   payload: {...}}       |                           |
  |<------------------------|                           |
```

这个时序展示了 Gateway 作为**中间人**的角色：它不仅负责消息路由，还在路由之前进行安全检查。Node Host 收到请求后执行实际操作，然后通过 `node.invoke.result` 将结果返回。

本节小结

1. **节点是远程能力的代理**——通过 WebSocket 连接到 Gateway，将设备的硬件和软件能力暴露给 Agent 使用。
2. **NodeSession 是节点的运行时快照**——包含身份标识、平台信息、能力列表和声明的命令集。
3. **星型拓扑简化了管理**——所有节点连接到同一个 Gateway，路由、权限、状态管理都集中在 Gateway 侧。

4. **平台决定默认角色**——iOS/Android 侧重移动感知能力（摄像头、定位），macOS/Linux 侧重系统执行能力（Shell、浏览器代理）。
5. **三层命令控制保证安全**——平台默认集 + 配置调整 → allowlist → 节点声明双向校验，白名单模式确保了最小权限原则。
6. **点分命名空间**组织命令——`canvas.*`、`camera.*`、`system.*` 等清晰划分能力域，便于精细化权限管理。